



Wait! Do this first.

Prerequisites Check

Terminal

```
$ docker --version
```

```
Docker version 24.0.7, build afdd53b
```

```
$ kind --version
```

```
kind version 0.20.0
```

```
$ kubectl version --client
```

```
Client Version: v1.28.2
```

✓ Expected Output

If you see version numbers, you're ready! If not, install the tools first.

📄 Installation Links



Docker Desktop

docker.com/products/docker-desktop



Kind (Kubernetes in Docker)

kind.sigs.k8s.io/docs/user/quick-start



kubectl

kubernetes.io/docs/tasks/tools

💡 Pro Tip

Docker Desktop now includes kubectl! You might already have everything you need.

The Problem: "It works on my machine"



Developer Laptop

Local Environment

- ✓ Python 3.9.7
- ✓ Node.js 16.3.0
- ✓ PostgreSQL 13.4
- ✓ macOS + Homebrew
- ✓ ENV vars in .bashrc

App Status

Running perfectly! ✓



Deploy



Production Server

Remote Environment

- ✗ Python 3.8.10
- ✗ Node.js 14.17.0
- ✗ PostgreSQL 12.8
- ✗ Ubuntu + apt-get
- ✗ ENV vars missing!

App Status

CRASH! ✖

“But it works on my machine!”

— Every Developer, Everywhere, Every Time

The Solution: Docker & The Shipping Container Metaphor

Standardize application packaging like shipping containers standardized global trade



Physical Shipping Container

- Standardized size & shape
- Works on any ship, truck, or train
- Contents are isolated & protected
- Global interoperability



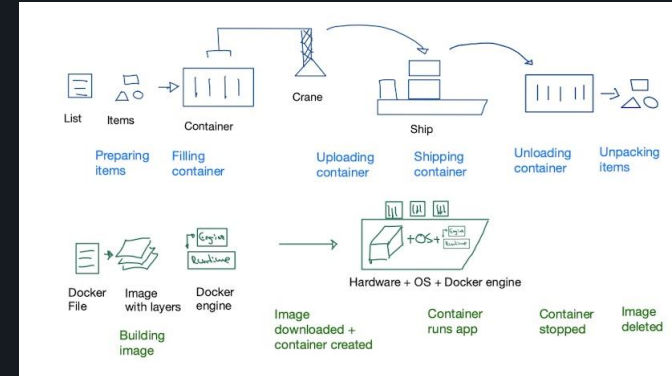
Docker Container

- Standardized packaging format
- Runs on any Linux/Windows/Mac
- App + dependencies isolated
- Deploy anywhere: laptop, server, cloud



Key Insight

Docker containers ensure your application behaves identically across all environments — development, testing, and production.



1

Package Everything

Code + runtime + libraries + config = One container

2

Ship Anywhere

Same container runs on dev laptop, staging, production

3

Run Consistently

"It works everywhere" — no more environment surprises

Dockerfile Anatomy

The blueprint for building your container



```
# Start with official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy dependencies first (for caching)
COPY requirements.txt .

# Install dependencies
RUN pip install -r requirements.txt

# Copy application code
COPY app.py .

# Run as non-root user (security)
USER appuser

# Command to run the app
CMD ["python", "app.py"]
```

FROM

Base image. Like choosing the OS for your container.

WORKDIR

Sets the working directory. All following commands run here.

COPY

Copies files from host to container filesystem.

RUN

Executes commands during build. Creates new layers.

CMD

Default command when container starts. Only one per Dockerfile.



Each instruction creates a **layer**. Layers are cached and reused!

`docker build -t myapp .`

Why Layer Caching Matters

Optimize build speed by leveraging Docker's layer cache

✗ Inefficient Approach

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app

# ✗ Copies EVERYTHING including app.py
COPY . .
RUN pip install -r requirements.txt
```

- ⚠ Any code change invalidates the cache
- ⚠ Dependencies reinstall on every build
- ⚠ Slow build times (2-5 minutes)

✓ Optimized Approach

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app

# ✓ Copy only requirements first
COPY requirements.txt .
RUN pip install -r requirements.txt

# ✓ Copy app code LAST
COPY app.py .
```

- ✓ Dependencies cached until requirements.txt changes
- ✓ Code changes only rebuild the last layer
- ✓ Fast builds (5-10 seconds)

2-5 min

Slow Build



Optimization

5-10 sec

Fast Build

Essential Docker Commands

The commands you'll use 90% of the time



docker build

Build an image

```
$ docker build -t myapp .
```

-t Tag the image

. Build context (current dir)

OUTPUT

Successfully tagged myapp:latest



docker run

Run a container

```
$ docker run -p 5000:5000 myapp
```

-p Port mapping

-d Detached mode

--name Name container

OUTPUT

Container started on port 5000



docker ps

List containers

```
$ docker ps
```

-a Show all (inc. stopped)

-q Quiet (IDs only)

OUTPUT

CONTAINER ID | IMAGE | STATUS

Stop

docker stop

Remove

docker rm

Logs

docker logs

Exec

docker exec -it bash

What is Kubernetes?

Container orchestration at scale — The Admiral of your container fleet



Kubernetes

Greek for "Helmsman" or "Pilot"

An open-source platform for automating deployment, scaling, and management of containerized applications.

- ✓ Auto-scaling
- ✓ Self-healing
- ✓ Load balancing
- ✓ Rolling updates

The Admiral Metaphor

-  **Containers** = Ships carrying cargo (apps)
-  **Kubernetes** = Admiral coordinating the fleet
-  **Control Plane** = Command center with maps & radar
-  **Worker Nodes** = Crew operating the ships

Kubernetes Architecture



Control Plane

- API Server — Entry point for all commands
- etcd — Database storing cluster state
- Scheduler — Decides where to run pods
- Controller Manager — Maintains desired state



Worker Nodes

- kubelet — Agent managing containers
- Container Runtime — Docker/containerd
- kube-proxy — Network routing



Your Apps

- Pods — Smallest deployable units
- Services — Expose apps to network
- Deployments — Manage pod replicas

Local K8s with Kind

Kubernetes in Docker — Run a full cluster on your laptop



What is Kind?

Kubernetes IN Docker

Kind runs Kubernetes clusters using Docker containers as nodes. Perfect for local development and CI/CD testing.

 Fast setup (< 2 min)

 Runs locally

 Multi-node support

 Easy cleanup

Installation

macOS/Linux

```
$brew install kind
```

Or download from GitHub releases

 kind.sigs.k8s.io

Create a Cluster

Terminal

```
$ kind create cluster \--config kind-config.yaml
```

Creating cluster "kind" ...

✓ Ensuring node image (kindest/node:v1.28.0)

✓ Preparing nodes ...

✓ Writing configuration ...

✓ Starting control-plane ...

Cluster creation complete!

VERIFY

```
$ kubectl cluster-info
```

Sample kind-config.yaml

```
kind: Cluster
```

```
apiVersion: kind.x-k8s.io/v1alpha4
```

```
nodes:
```

```
- role: control-plane
```

```
- role: worker
```

```
- role: worker
```


Deployment Manifest

Define how your application runs with declarative YAML

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
spec:
  # Number of pod replicas
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myapp:latest
          ports:
            - containerPort: 5000
```

1 apiVersion & kind

Specifies the Kubernetes API version and resource type (Deployment).

2 metadata

Names and labels for identifying the deployment. Labels are key-value pairs.

3 replicas: 2

Run **2 identical pods** for high availability. If one fails, the other handles traffic.

4 template

Defines the pod spec: which container image to run and what ports to expose.



Deployment manages ReplicaSet → ReplicaSet manages Pods

kubectl apply -f deployment.yaml

Service Manifest

Expose your application to the network

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  # Service type
  type: NodePort
  selector:
    # Match pods with this label
    app: myapp
  ports:
    # Port configuration
    - port: 80
      targetPort: 5000
      # External port (30000-32767)
      nodePort: 30000
```

Service Types

ClusterIP Internal only (default)

NodePort Expose on each node's IP

LoadBalancer Cloud provider LB

🔌 nodePort: 30000

Exposes the service on a static port (30000-32767) on each node's IP.

Access your app at:
<http://localhost:30000>

Port Mapping

External	→	Service	→	Container
:30000		:80		:5000



Services provide **load balancing** across pods automatically!

`kubectl apply -f service.yaml`

The Kubectl Bible

Essential commands for Kubernetes operations



kubectl apply

Create/Update resources

```
$ kubectl apply -f deployment.yaml  
deployment.apps/myapp created
```

- Applies YAML configurations
- Creates if not exists
- Updates if exists

`-f = --filename`



kubectl get

List resources

```
$ kubectl get pods  
NAME READY STATUS  
myapp-xxx 1/1 Running
```

- pods, services, deployments
- nodes, events, configmaps
- Wide output with `-o wide`

`-w = watch mode`



kubectl logs

View pod logs

```
$ kubectl logs -f myapp-xxx  
[INFO] Server started on port 5000  
[INFO] Connected to database
```

- Stream logs with `-f`
- Previous container with `--previous`
- All containers with `--all-containers`

`-f = follow (tail)`

Describe

`kubectl describe pod`

Exec

`kubectl exec -it -- bash`

Delete

`kubectl delete -f file.yaml`

Port Forward

`kubectl port-forward svc/app 8080:80`

Why CI/CD?

Automation is a superpower



Manual Deployment

- ✗ Build artifacts locally
- ✗ SSH into production server
- ✗ Copy files manually (SCP/FTP)
- ✗ Restart services by hand
- ✗ Hope nothing breaks 🙄

Result: Error-prone, slow, scary



CI/CD Automation

- ✓ Code push triggers build automatically
- ✓ Run tests in isolated environment
- ✓ Build & push container image
- ✓ Deploy to Kubernetes automatically
- ✓ Rollback if something goes wrong

Result: Fast, reliable, repeatable

30+ min

Per deployment

High

Error rate

Weekly

Deploy frequency

< 5 min

Per deployment

Low

Error rate

Multiple/day

Deploy frequency



"Automation is a superpower"

CI/CD transforms deployment from a scary manual task into a reliable, automated process.

GitHub Actions & GitHub Container Registry

Build, test, and deploy directly from your GitHub repository



GitHub Actions

CI/CD built into GitHub



Event-Driven

Triggers on push, PR, schedule, or manual dispatch



Matrix Builds

Test on multiple OS, languages, or versions in parallel



Marketplace

Thousands of pre-built actions for common tasks



Free Tier

2,000 minutes/month for public repositories



GitHub Container Registry

ghcr.io — Docker images hosted on GitHub



Integrated with GitHub

Same permissions as your repository. No separate login needed.



Image Tagging

Tag images with commit SHA, branch name, or semantic version



Security Scanning

Automatic vulnerability scanning for container images

Image URL Format

`ghcr.io/OWNER/IMAGE_NAME:TAG`

Example: `ghcr.io/firojpaudel/myapp:v1.0.0`



Everything in One Place

Code, issues, pull requests, CI/CD, and container registry — all on GitHub

The Workflow YAML

Build → Tag → Push cycle automated

.github/workflows/docker-build.yml

```
name: Build & Push Docker Image
on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # 1. Checkout code
      - uses: actions/checkout@v4
      # 2. Login to GHCR
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      # 3. Build & Push
      - run: |
          docker build -t ghcr.io/${{ github.repository }}:latest .
          docker push ghcr.io/${{ github.repository }}:latest
```

Workflow Structure

- 1 **name** — Workflow identifier
- 2 **on** — Trigger events
- 3 **jobs** — Parallel or sequential tasks
- 4 **steps** — Individual commands/actions

Build → Tag → Push

- 1 **Build** image from Dockerfile
- 2 **Tag** with registry & version
- 3 **Push** to GHCR

Pro Tip

Use commit SHA for immutable tags: :sha-7a8b9c



Workflow runs automatically on every push to main branch

[Actions tab → View logs](#)

Permissions: GITHUB_TOKEN

Why your workflow needs write access



What is GITHUB_TOKEN?

Auto-generated authentication token

GitHub automatically creates a token for each workflow run. It authenticates the workflow to GitHub's API and services.

In your workflow:

```
${{ secrets.GITHUB_TOKEN }}
```



The Problem

By default, GITHUB_TOKEN has **read-only** permissions for security. But pushing Docker images requires **write** access to packages.

Error: denied

unauthorized: unauthenticated



The Solution

Grant **write** permissions to packages in your workflow:

```
name: Build & Push
# Add permissions block
permissions:
  contents: read
  packages: write
jobs:
  build:
    runs-on: ubuntu-latest
    steps: ...
```

Permission Levels

contents	read
packages	write
actions	read



Security Best Practice

Grant only the minimum permissions needed. Use **permissions:** block instead of repository-wide settings.

Multi-Stage Builds

Optimize image size with the Java Example

Dockerfile (Multi-Stage)

```
# ===== BUILD STAGE =====
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
# Build the JAR file
RUN mvn clean package -DskipTests

# ===== RUN STAGE =====
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
# Copy ONLY the JAR from builder
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build Stage

 Full JDK + Maven

 Source code

 Compile & package

~700 MB

Image size

Run Stage

 Minimal JRE only

 Compiled JAR only

 Run the app

~80 MB

Image size

700 MB

Single-stage



Multi-stage

80 MB

90% smaller!

Scaling Demo

Scale from 2 to 10 replicas in seconds

>_ Scale Command

Terminal

```
$ kubectl scale deployment myapp-deployment --replicas=10
deployment.apps/myapp-deployment scaled
```

ALTERNATIVE

```
kubectl edit deployment myapp-deployment
```

Pod Visualization



Verify Scaling

Watch pods being created

```
$ kubectl get pods -w
NAME READY STATUS
myapp-xxx-1 1/1 Running
myapp-xxx-2 1/1 Running
myapp-xxx-3 0/1 ContainerCreating
myapp-xxx-4 0/1 ContainerCreating
...
myapp-xxx-10 1/1 Running
```

Auto-Scaling (HPA)

Horizontal Pod Autoscaler automatically scales based on CPU/memory usage:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    name: myapp-deployment
  minReplicas: 2
  maxReplicas: 20
```



Kubernetes Magic

Scale from 2 to 10 replicas in seconds. The Service automatically load-balances traffic across all pods.

Summary Checklist

What you learned in 150 minutes



Containers

Docker Fundamentals

- ✓ **Dockerfile**
Build images with FROM, COPY, RUN, CMD
- ✓ **Layer Caching**
Optimize builds with smart COPY ordering
- ✓ **Commands**
docker build, run, ps, logs, exec
- ✓ **Multi-Stage**
Reduce image size with build/run stages



Clusters

Kubernetes Orchestration

- ✓ **Kind**
Run local K8s clusters with Docker
- ✓ **Deployments**
Define pod replicas and updates
- ✓ **Services**
Expose apps with NodePort
- ✓ **kubectl**
apply, get, logs, scale



Pipelines

CI/CD Automation

- ✓ **GitHub Actions**
Event-driven workflows on push/PR
- ✓ **GHCR**
Host Docker images on GitHub
- ✓ **Build → Tag → Push**
Automated container lifecycle
- ✓ **Permissions**
GITHUB_TOKEN with write access



You Did It!

From **Zero** to **Hero**

Code → Container → Cluster → Cloud



Docker



K8s



CI/CD

 Workshop Complete

Questions & Answers

What would you like to explore further?



Workshop Repository

All code and examples

github.com/Firojpaudel

/docker-k8s-ci-cd-demo

- ✓ Complete Dockerfile examples
- ✓ Kubernetes manifests
- ✓ GitHub Actions workflows
- ✓ Step-by-step instructions



Additional Resources

Continue your learning



Docker Docs

docs.docker.com



Kubernetes Docs

kubernetes.io/docs



GitHub Actions

docs.github.com/actions



Kind Quick Start

kind.sigs.k8s.io



firojpaudel@gmail.com



/in/firoj-paudel-17a66724b/

Thank you for joining!